

Section A — Concept Application Answers

Food Delivery Domain | Java • M6 - Spring • M6-A1

S1 — Session Tracking & Cart

For this cart feature, I would choose **HttpSession**. The cart must remain available while the user moves between restaurant, menu, cart and checkout pages, and login is not required at this stage. HttpSession keeps the cart on the server and associates it with the user session.

I would avoid **URL rewriting** as the primary approach because cart state or a session identifier would have to be carried through many URLs. This makes URLs longer, can expose session information in browser history and logs, and is harder to maintain when there are many pages and links.

API calls: Add the cart with `session.setAttribute("cart", cart)`. Retrieve it with `session.getAttribute("cart")`. Clear only the cart with `session.removeAttribute("cart")`. If the entire session must be destroyed, use `session.invalidate()`.

S2 — Hibernate Restaurant–MenuItem Mapping

I would use a **bidirectional one-to-many / many-to-one** relationship. One Restaurant has many MenuItems, while each MenuItem belongs to exactly one Restaurant. The MenuItem side owns the foreign key.

Restaurant: `@OneToMany(mappedBy="restaurant", cascade=CascadeType.ALL, fetch=FetchType.LAZY)` with a `List<MenuItem> menuItems`.

MenuItem: `@ManyToOne(fetch=FetchType.LAZY)` and `@JoinColumn(name="restaurant_id", nullable=false)` with a Restaurant field.

With **CascadeType.ALL**, operations on the Restaurant are cascaded to its associated MenuItems. Therefore, deleting a Restaurant can also delete its MenuItems. This is appropriate when MenuItems are dependent on that Restaurant and should not exist independently.

I would normally choose **LAZY** loading for the collection because a restaurant list may contain many menu records and the application may not always need them. A **LazyInitializationException** can happen in a Servlet application when the DAO closes the Hibernate Session and a JSP or controller later accesses the lazy collection. The collection should be initialized while the Session is open, fetched in the query, or converted to a DTO before the Session closes.

S3 — DAO Pattern for Order Management

Putting JDBC directly inside OrderServlet creates two important problems.

1. Maintainability: HTTP handling becomes mixed with SQL, Connection, PreparedStatement, exception handling and resource cleanup. A database change makes the servlet harder to understand, test and modify.

2. Scalability and reuse: Other controllers may duplicate the same database code. A change to connection handling or SQL then has to be repeated in several places, increasing duplication and inconsistency.

The **DAO pattern** moves persistence operations into an OrderDAO such as `placeOrder()`, `getOrderById()` and `getAllOrders()`. The servlet focuses on HTTP while the DAO focuses on database access.

This also makes a future JDBC-to-Hibernate migration easier. Controllers can continue using the DAO interface while the implementation changes from JDBC PreparedStatement to Hibernate Session/HQL. The web layer does not need to know which persistence technology is underneath.

S4 — Singleton, Connection Pool & Thread Safety

Under load, I would use a **Singleton database manager** to create and hold one shared **DataSource/connection pool**, rather than opening a new physical connection for every request. I would not normally share one JDBC Connection between all requests because concurrent transactions and connection state must remain isolated.

Typical structure:

```
private static volatile DBManager instance;

private DBManager() { /* create/configure DataSource */ }

public static DBManager getInstance() {
    if (instance == null) {
        synchronized (DBManager.class) {
            if (instance == null) instance = new DBManager();
        }
    }
    return instance;
}
```

The Servlet container is multithreaded, so two requests can execute at the same time. Without synchronization, two threads could both see a null instance and create two Singleton objects. Using synchronized initialization, a static-holder Singleton, or another thread-safe initialization method prevents this race. Each request should borrow a connection from the pool and return it afterward.

S5 — Spring IoC & Dependency Injection

If six controllers each call **new OrderService()**, object creation is duplicated and the controllers are tightly coupled to a concrete implementation. If OrderService later needs a repository, transaction manager or other dependency, all six controllers may need manual changes.

Spring **IoC** solves this by creating and managing the OrderService bean and injecting it into controllers. The controllers declare what they need instead of deciding how to construct it.

Setter injection:

```
@Autowired
public void setOrderService(OrderService orderService) {
    this.orderService = orderService;
}
```

Constructor injection:

```
private final OrderService orderService;

@Autowired
public OrderController(OrderService orderService) {
    this.orderService = orderService;
}
```

I would prefer **constructor injection** for OrderController because OrderService is a required dependency. It makes the dependency explicit, supports final fields, and makes unit testing straightforward. Setter injection is useful when a dependency is optional or intentionally replaceable after construction.

S6 — Spring MVC POST 404 & Request Lifecycle

The request flow is: **Client/Postman** → **DispatcherServlet** → **HandlerMapping** → **Controller method** → **Service/DAO** → **ModelAndView or redirect** → **ViewResolver** → **JSP** → **HTTP response**.

The web container first maps the incoming request to the DispatcherServlet using web.xml.

DispatcherServlet receives the request and asks **HandlerMapping** to find a controller handler whose URL and HTTP method match. Spring then invokes that method. A ModelAndView goes through the ViewResolver to locate the JSP; a redirect sends a redirect response so the client makes a new request.

For a 404 on POST /orders/place, I would check at least these points: **(1)** web.xml DispatcherServlet mapping, normally /; **(2)** component scanning so the package containing the @Controller is detected; **(3)** the request method and mapping in Postman; and **(4)** the deployed application context path, because the actual URL may be /ApplicationName/orders/place.

HandlerMapping connects the incoming request to the matching controller handler. If the class has @RequestMapping("/orders") and the method has @PostMapping("/place"), the combined mapping is **POST /orders/place**. A wrong class mapping, wrong method mapping, missing component scan, or incorrect DispatcherServlet configuration can prevent the handler from being found and lead to a 404.